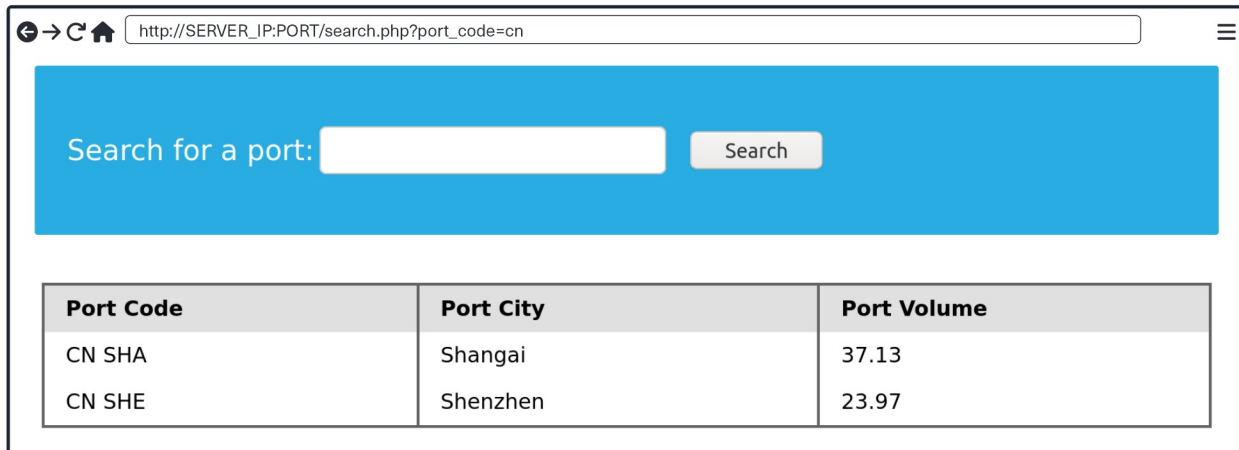


Union Injection

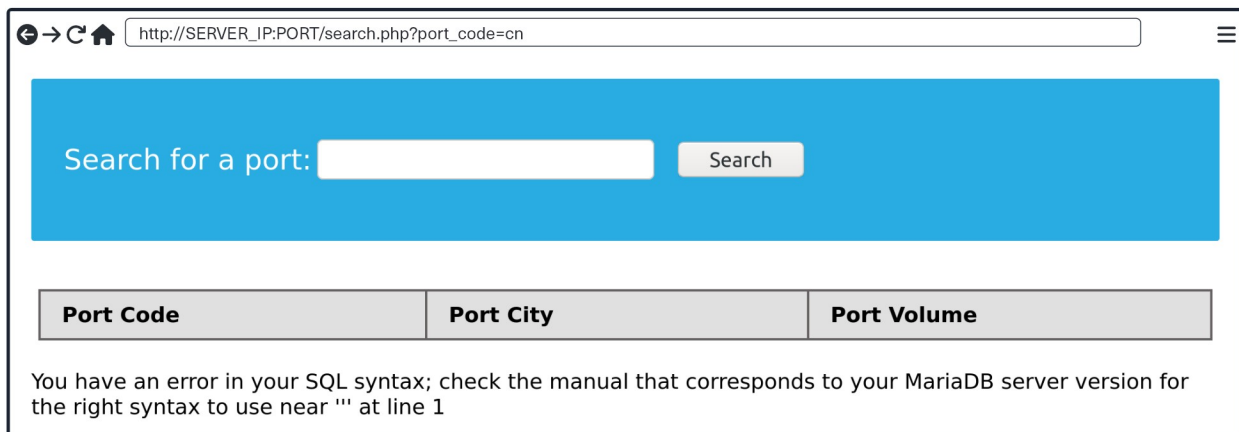
Now that we know how the Union clause works and how to use it let us learn how to utilize it in our SQL injections. Let us take the following example:



The screenshot shows a web browser window with the URL `http://SERVER_IP:PORT/search.php?port_code=cn`. The page has a blue header with the text "Search for a port:" followed by a text input field and a "Search" button. Below the header is a table with three columns: "Port Code", "Port City", and "Port Volume". The table contains two rows of data.

Port Code	Port City	Port Volume
CN SHA	Shangai	37.13
CN SHE	Shenzhen	23.97

We see a potential SQL injection in the search parameters. We apply the SQLi Discovery steps by injecting a single quote ('), and we do get an error:



The screenshot shows the same web browser window as before, but the search input field now contains a single quote ('). The "Search" button is still present. Below the header, the table is still visible, but an error message is displayed below it.

You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near ''' at line 1

Since we caused an error, this may mean that the page is vulnerable to SQL injection. This scenario is ideal for exploitation through Union-based injection, as we can see our queries' results.

Detect number of columns

Before going ahead and exploiting Union-based queries, we need to find the number of columns selected by the server. There are two methods of detecting the number of columns:

- Using `ORDER BY`
- Using `UNION`

Using ORDER BY

Integrated Terminal

```
[eu-academy-3]-[10.10.14.24]-[htb-ac-819475@htb-dvvpal7wif]-[~]  
[*]$  
Specified does not exist.
```

For example, we can start with `order by 1`, sort by the first column, and succeed, as the table must have at least one column. Then we will do `order by 2` and then `order by 3` until we reach a number that returns an error, or the page does not show any output, which means that this column number does not exist. The final successful column we successfully sorted by gives us the total number of columns.

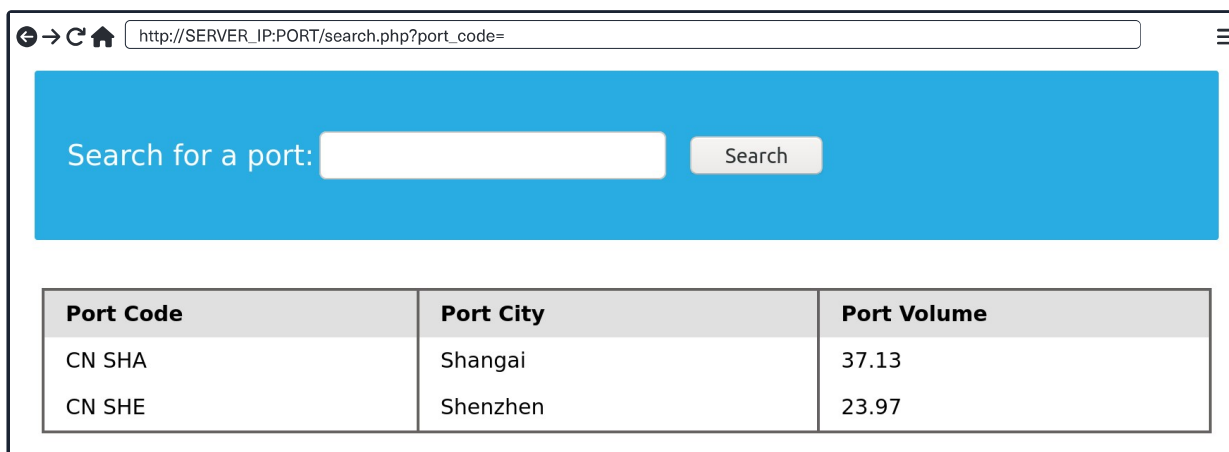
If we failed at `order by 4`, this means the table has three columns, which is the number of columns we were able to sort by successfully. Let us go back to our previous example and attempt the same, with the following payload:

Code: `sql`

```
' order by 1-- -
```

Reminder: We are adding an extra dash (-) at the end, to show you that there is a space after (--).

As we see, we get a normal result:



The screenshot shows a web browser with the address bar containing `http://SERVER_IP:PORT/search.php?port_code=`. The page has a blue header with the text "Search for a port:" followed by an input field and a "Search" button. Below the header is a table with three columns: "Port Code", "Port City", and "Port Volume". The table contains two rows of data.

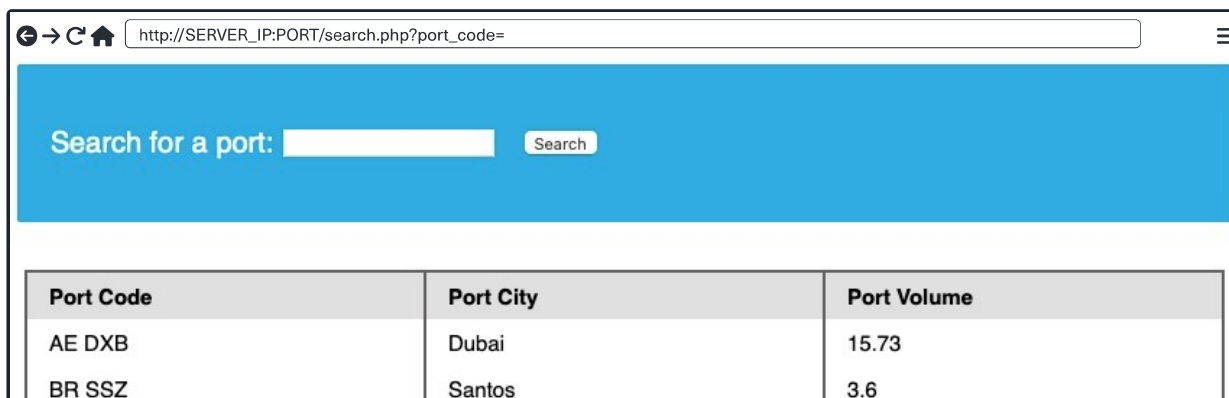
Port Code	Port City	Port Volume
CN SHA	Shanghai	37.13
CN SHE	Shenzhen	23.97

Next, let us try to sort by the second column, with the following payload:

Code: `sql`

```
' order by 2-- -
```

We still get the results. We notice that they are sorted differently, as expected:

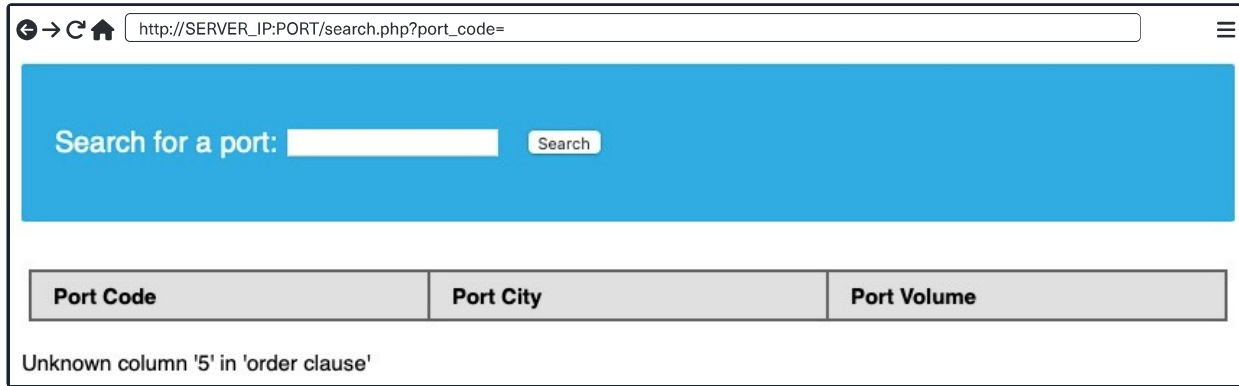


The screenshot shows the same web browser interface as before, but the table data is now sorted by the "Port City" column. The table contains two rows of data.

Port Code	Port City	Port Volume
AE DXB	Dubai	15.73
BR SSZ	Santos	3.6

Integrated Terminal

We do the same for column 3 and 4 and get the results back. However, when we try to **ORDER BY** column 5, we get the following error:



The screenshot shows a web browser window with the URL `http://SERVER_IP:PORT/search.php?port_code=`. The page has a blue header with the text "Search for a port:" followed by a text input field and a "Search" button. Below the header is a table with three columns: "Port Code", "Port City", and "Port Volume". At the bottom of the page, an error message is displayed: "Unknown column '5' in 'order clause'".

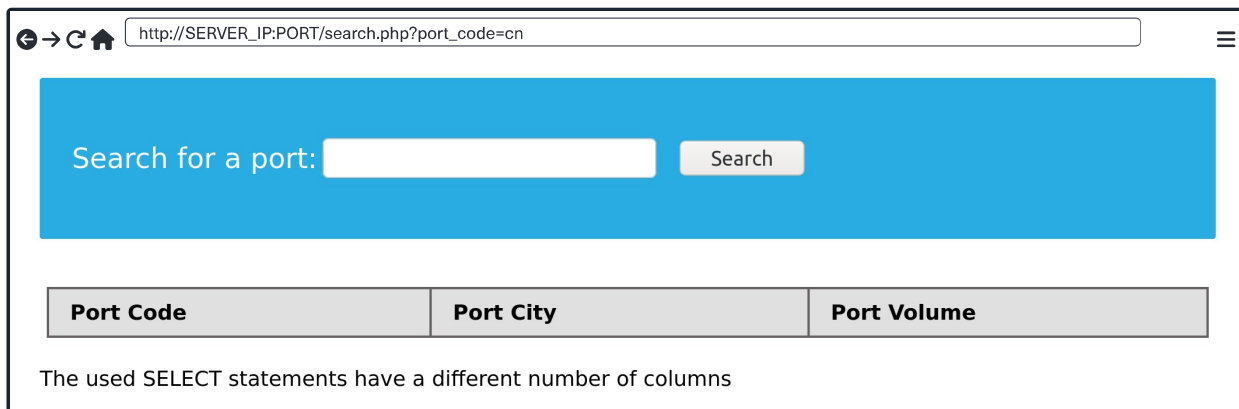
This means that this table has exactly 4 columns .

Using UNION

The other method is to attempt a Union injection with a different number of columns until we successfully get the results back. The first method always returns the results until we hit an error, while this method always gives an error until we get a success. We can start by injecting a 3 column **UNION** query:

```
Code: sql
cn' UNION select 1,2,3-- -
```

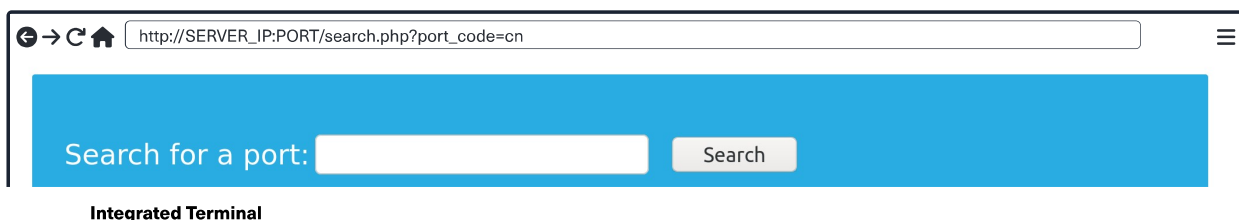
We get an error saying that the number of columns don't match:



The screenshot shows the same web browser window as before, but with the URL `http://SERVER_IP:PORT/search.php?port_code=cn`. The error message at the bottom of the page now reads: "The used SELECT statements have a different number of columns".

So, let's try four columns and see the response:

```
Code: sql
cn' UNION select 1,2,3,4-- -
```



The screenshot shows the same web browser window, but with the URL `http://SERVER_IP:PORT/search.php?port_code=cn`. The error message is gone, and the page displays the "Search for a port:" form and the table with three columns: "Port Code", "Port City", and "Port Volume". Below the table, the text "Integrated Terminal" is visible.

Port Code	Port City	Port Volume
2	3	4

This time we successfully get the results, meaning once again that the table has 4 columns. We can use either method to determine the number of columns. Once we know the number of columns, we know how to form our payload, and we can proceed to the next step.

Location of Injection

While a query may return multiple columns, the web application may only display some of them. So, if we inject our query in a column that is not printed on the page, we will not get its output. This is why we need to determine which columns are printed to the page, to determine where to place our injection. In the previous example, while the injected query returned 1, 2, 3, and 4, we saw only 2, 3, and 4 displayed back to us on the page as the output data:

The screenshot shows a web browser window with the URL `http://SERVER_IP:PORT/search.php?port_code=cn`. The page has a blue header with the text "Search for a port:" followed by an input field and a "Search" button. Below the header is a table with three columns: "Port Code", "Port City", and "Port Volume". The table contains one row of data: "2", "3", and "4".

It is very common that not every column will be displayed back to the user. For example, the ID field is often used to link different tables together, but the user doesn't need to see it. This tells us that columns 2 and 3, and 4 are printed to place our injection in any of them. **We cannot place our injection at the beginning, or its output will not be printed.**

This is the benefit of using numbers as our junk data, as it makes it easy to track which columns are printed, so we know at which column to place our query. To test that we can get actual data from the database 'rather than just numbers,' we can use the `@@version` SQL query as a test and place it in the second column instead of the number 2:

Code: `sql`

```
cn' UNION select 1,@@version,3,4-- -
```

The screenshot shows the same web browser window as before, but the table now displays the results of the SQL injection. The table has three columns: "Port Code", "Port City", and "Port Volume". The first row of data is: "1", "cn' UNION select 1,@@version,3,4-- -", and "4".

Integrated Terminal

As we can see, we can get the database version displayed. Now we know how to form our Union SQL injection payloads to successfully get the output of our query printed on the page. In the next section, we will discuss how to enumerate the database and get data from other tables and databases.



Connect to Pwnbox

Your own web-based Parrot Linux instance to play our labs.

Pwnbox Location

AU

122ms

Terminate Pwnbox to switch location

```
10649 |
10650 |
10651 |
10652 |
10653 |
10654 |
d001 |
d002 |
d003 |
d004 |
d005 |
d006 |
d007 |
d008 |
d009 |
+-----+
663 rows in set (0.248 sec)
MariaDB [employees]>
```

Full Screen

Terminate

Reset

Life Left: 87m

+



Connected to htb-dvvpal7wif:1 (htb-ac-819475)

Questions

Answer the question(s) below to complete this Section and earn cubes!



Cheat Sheet

Target(s): [Click here to spawn the target system!](#)

+0

Use a Union injection to get the result of 'user()'

Submit your answer here...

Integrated Terminal

[< Previous](#)[Next >](#)[📄 Cheat Sheet](#)[? Go to Questions](#)

Table of Contents

[Introduction](#)

Databases

[Intro to Databases](#)[Types of Databases](#)

MySQL

[📦 Intro to MySQL](#)[📦 SQL Statements](#)[📦 Query Results](#)[📦 SQL Operators](#)

SQL Injections

[Intro to SQL Injections](#)[📦 Subverting Query Logic](#)[📦 Using Comments](#)[📦 Union Clause](#)[📦 Union Injection](#)

Exploitation

[📦 Database Enumeration](#)[📦 Reading Files](#)[📦 Writing Files](#)

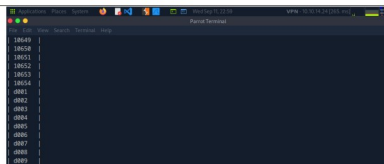
Mitigations

[Mitigating SQL Injection](#)

Closing it Out

[📦 Skills Assessment - SQL Injection Fundamentals](#)

My Workstation

**Integrated Terminal**

 Interact	 Terminate	 Reset	Life Left: 87m	
--	---	---	----------------	---

Integrated Terminal
